NOM Prénom : Ethan Georget
BTS SIO SISR - Bloc 2
Compte-Rendu : Réplication de données (Asynchrone Pull & Synchrone GlusterFS)

DOSSIER TECHNIQUE : STRATÉGIES DE RÉPLICATION POUR LA HAUTE DISPONIBILITÉ

1. Présentation et préparation de l'environnement

L'objectif de ce laboratoire est de mettre en œuvre différentes techniques de réplication de données. Associées à des systèmes de fail-over ou de répartition de charge, ces méthodes permettent d'assurer la haute disponibilité de services dynamiques (bases de données, serveurs web).

L'infrastructure repose sur deux serveurs web : srv-web (192.168.200.51) et srv-web-bis (192.168.200.52). Le déploiement se fait en deux étapes : une réplication asynchrone par batch et une réplication synchrone en temps réel.

1.2 Validation de la connectivité réseau

Avant de mettre en place la réplication, j'ai vérifié la stabilité de l'infrastructure en effectuant des tests de connectivité (ping) depuis la machine cliente vers les deux futurs serveurs de données (192.168.200.51 et 52). J'ai également validé que le service Apache était opérationnel sur chaque nœud en accédant aux pages d'index personnalisées via le navigateur.

```
PING 192.168.200.52 (192.168.200.52) 56(84) bytes of data.
64 bytes from 192.168.200.52: icmp_seq=1 ttl=64 time=0.827 ms
64 bytes from 192.168.200.52: icmp_seq=2 ttl=64 time=0.283 ms
64 bytes from 192.168.200.52: icmp_seq=3 ttl=64 time=0.246 ms
```

2. Réplication asynchrone par mode Pull (Rsync)

En mode asynchrone, les données sont recopiées à intervalles réguliers. Le serveur esclave (srv-web-bis) vient 'tirer' les modifications présentes sur le maître (srv-web). Cette méthode est adaptée aux services statiques où les modifications sont rares.

2.1 Configuration de rsync et automatisation

Sur le maître, le démon rsync est configuré via /etc/rsyncd.conf pour exposer le module 'donnees' correspondant au répertoire /var/www/html/. Sur l'esclave, nous utilisons l'ordonnanceur 'cron' pour déclencher la synchronisation chaque minute via une crontab.

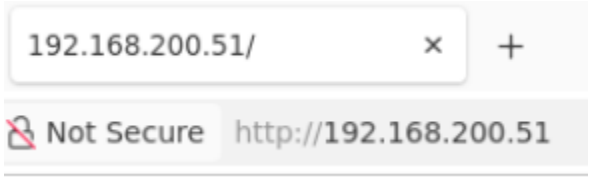
Pour valider le fonctionnement du module, j'ai lancé une première synchronisation manuelle depuis le serveur esclave. J'ai pu confirmer le succès du transfert en constatant que les fichiers « essai1 » et « essai2 », créés sur le maître, étaient instantanément répliqués dans le répertoire /var/www/html/ du serveur secondaire.

```
rsync -a --delete-after 192.168.200.51::donnees /var/www/html/
```

Dans le répertoire /var/www/html :

```
index.html  index.php  mutillidae
```

Afin de tester l'automatisation, j'ai modifié l'index du serveur maître. Après un délai inférieur à une minute, la tâche Cron a déclenché la synchronisation, rendant les deux sites strictement identiques sur le client. Ce test valide la fiabilité du mode « Pull » pour des données statiques.



3. Réplication synchrone en temps réel (GlusterFS)

La réplication synchrone permet d'éliminer les délais de propagation. GlusterFS crée un système de fichiers distribué où chaque modification sur un nœud est instantanément répercutée sur l'autre, supprimant tout état d'incohérence.

3.1 Préparation du stockage physique

Un disque dur supplémentaire de 1 Go (/dev/sdb) a été ajouté à chaque serveur. Nous avons procédé au partitionnement (fdisk), au formatage en ext4 et au montage persistant via le fichier /etc/fstab en utilisant les identifiants UUID des disques.

```
UUID="0a60394c-..... /data/glusterfs ext4 defaults 0 0
```

Sur chaque serveur, j'ai utilisé l'utilitaire fdisk pour partitionner le nouveau disque /dev/sdb. J'ai créé une partition primaire occupant tout l'espace disque, que j'ai ensuite formatée avec le système de fichiers EXT4 pour accueillir les briques de données GlusterFS avec la commande : **mkfs.ext4 /dev/sdb1**

```
mke2fs 1.47.0 (5-Feb-2023)
Creating filesystem with 261888 4k blocks and 65536 inodes
Filesystem UUID: cfe8092d-02c9-4097-9c2a-6a0c8edd2c11
Superblock backups stored on blocks:
        32768, 98304, 163840, 229376

Allocating group tables: done
Writing inode tables: done
Creating journal (4096 blocks): done
Writing superblocks and filesystem accounting information: done
```

3.2 Initialisation du cluster et du volume répliqué

La mise en grappe s'effectue avec la commande 'gluster peer probe'. Nous avons ensuite créé un volume répliqué nommé 'glusterfsvolume' (replica 2) englobant les briques de stockage des deux serveurs. Le volume est ensuite monté localement sur /mnt/glusterfsvolume.

/mnt/glusterfsvolume# gluster peer status:

```
Number of Peers: 1

Hostname: srv-web-bis.mlif.local
Uuid: 3c96c402-1153-42c7-ae92-33ec2bb584b7
State: Peer in Cluster (Connected)
root@srv-web:/mnt/glusterfsvolume# gluster pool list
UUID                                    Hostname                State
3c96c402-1153-42c7-ae92-33ec2bb584b7    srv-web-bis.mlif.local  Connected
4cf0cd09-1cc3-48df-bf04-d718a09e5617    localhost               Connected
root@srv-web:/mnt/glusterfsvolume#
```

/mnt/glusterfsvolume# gluster volume info glusterfsvolume:

```
Volume Name: glusterfsvolume
Type: Replicate
Volume ID: 2c4b7907-0b52-4845-8509-e6aa4d0a5117
Status: Started
Snapshot Count: 0
Number of Bricks: 1 x 2 = 2
Transport-type: tcp
Bricks:
Brick1: srv-web.mlif.local:/data/glusterfs/glustervolume
Brick2: srv-web-bis.mlif.local:/data/glusterfs/glustervolume
Options Reconfigured:
nfs.disable: on
performance.readdir-ahead: on
transport.address-family: inet
```

La dernière étape a consisté à monter le volume répliqué sur le point de montage /mnt/glusterfsvolume. J'ai vérifié la réplication en temps réel en créant un fichier sur le premier nœud et en constatant sa disponibilité immédiate sur le second, prouvant l'absence de délai de propagation propre au mode synchrone

```
root@srv-web-ethan:/mnt/glusterfsvolume# ls
test  test2
```

4. Bilan et comparaison

La réplication synchrone via GlusterFS s'avère bien plus performante pour la haute disponibilité active/active, car elle permet des écritures simultanées sur les deux nœuds sans perte de données. Couplée à un répartiteur de charge, elle assure une continuité de service optimale même en cas de panne d'un serveur.